

ResNet Machine vision models using FACET before and after adversarial debiasing

Normal model training

Imports and run time set up

Since this notebook is ment to run on all systems we have to first check the status of the hardware and set the model training accordingly.

```
In [1]: import os
import pandas as pd
import numpy as np
from PIL import Image

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import models, transforms

from sklearn.preprocessing import LabelEncoder
from collections import defaultdict
from tqdm import tqdm

import torch
print("torch.version:", torch.__version__)
print("cuda available:", torch.cuda.is_available())
print("cuda device count:", torch.cuda.device_count())
print("cuda version (compiled):", torch.version.cuda)
print("device name 0:", torch.cuda.get_device_name(0) if torch.cuda.is_available())

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
torch.version: 2.5.1+cu121
cuda available: True
cuda device count: 1
cuda version (compiled): 12.1
device name 0: NVIDIA GeForce RTX 3060
Using device: cuda
```

```
In [ ]: # Configuration variables
CSV_PATH = "./annotations.csv"
IMAGES_ROOT = "../Data/"
BATCH_SIZE = 64
NUM_WORKERS = 4
NUM_EPOCHS = 10
LR = 1e-4
```

```
VAL_SPLIT = 0.15
TEST_SPLIT = 0.15
```

In order to train the model we must check to see if the facet dataset images are present. Otherwise we won't be able to train the model.

```
In [3]: from pathlib import Path

# Point this at the parent of imgs_1, imgs_2, imgs_3
BASE = Path("../Data/").resolve()

IMAGE_DIRS = [
    BASE / "imgs_1",
    BASE / "imgs_2",
    BASE / "imgs_3",
]

# Quick sanity check
for d in IMAGE_DIRS:
    print("Checking", d, "exists:", d.exists())

# Build mapping from bare filename to full path
filename_to_path = {}

for d in IMAGE_DIRS:
    for p in d.glob("*"):
        if p.is_file():
            fname = p.name # just 'abc.jpg', no directory
            if fname in filename_to_path:
                # Optional: warn if duplicates
                print("WARNING: duplicate filename across dirs:", fname)
            filename_to_path[fname] = str(p)

print("Total unique filenames indexed:", len(filename_to_path))
```

```
Checking C:\Users\Lucas\Desktop\Development\FairMachine\Data\imgs_1 exists: True
Checking C:\Users\Lucas\Desktop\Development\FairMachine\Data\imgs_2 exists: True
Checking C:\Users\Lucas\Desktop\Development\FairMachine\Data\imgs_3 exists: True
Total unique filenames indexed: 31702
```

Label encodings

In order to perform adversarial bias mitigation and to allow for simple metric analysis the skin tone categories have been compressed into "light" and "dark".

```
In [4]: df = pd.read_csv(CSV_PATH)

# Encode class1 as integer labels
label_encoder = LabelEncoder()
df["label_idx"] = label_encoder.fit_transform(df["class1"].astype(str))
num_classes = df["label_idx"].nunique()
print("Num classes:", num_classes)

skin_cols = [f"skin_tone_{i}" for i in range(1, 11)] # skin_tone_1 ... skin_tone_10
```

```
def row_to_skin_group(row):
    # If row has 1-hot skin tone cols, find which one is 1
    skin_values = row[skin_cols].values
    if skin_values.sum() == 0:
        return 2 # NA group

    tone = np.argmax(skin_values) + 1 # 1-10

    # Light vs dark threshold from your EDA (1-5 vs 6-10)
    if tone <= 5:
        return 0 # light
    else:
        return 1 # dark

df["skin_idx"] = df.apply(row_to_skin_group, axis=1)
print(df["skin_idx"].value_counts())
```

Num classes: 52

skin_idx

0 35538

1 9395

2 4618

Name: count, dtype: int64

Helpers

```
In [5]: import os
from PIL import Image
from torch.utils.data import Dataset
import torch

class FacetDataset(Dataset):
    def __init__(self, df, image_roots, transform=None):
        """
        df: DataFrame with columns 'filename', 'label_idx', 'skin_idx'
        image_roots: str or list[str] of directories containing images
                     e.g. ["data/files/imgs_1", "data/files/imgs_2", "data/files/im

        """
        self.df = df.reset_index(drop=True)

        # Normalize to a list
        if isinstance(image_roots, str):
            image_roots = [image_roots]
        self.image_roots = image_roots

        self.transform = transform

        # Build an index: filename -> full path
        self.filename_to_path = {}
        for root in self.image_roots:
            for dirpath, _, filenames in os.walk(root):
                for fname in filenames:
                    # If duplicate names exist in different dirs, last one wins.
                    full_path = os.path.join(dirpath, fname)
```

```

        self.filename_to_path[fname] = full_path

    if not self.filename_to_path:
        raise RuntimeError(
            f"No images found under roots: {self.image_roots}. "
            "Check that the paths are correct."
        )

    print(
        f"Indexed {len(self.filename_to_path)} image files from roots: "
        f"{self.image_roots}"
    )

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        filename = row["filename"]

        # Try exact key first
        img_path = self.filename_to_path.get(filename)

        # If the CSV includes a relative path like 'imgs_1/foo.jpg',
        # fall back to the basename
        if img_path is None:
            base = os.path.basename(filename)
            img_path = self.filename_to_path.get(base)

        if img_path is None:
            raise FileNotFoundError(
                f"{filename} not found in indexed image paths. "
                f"Example keys: {list(self.filename_to_path.keys())[:5]}"
            )

        image = Image.open(img_path).convert("RGB")
        if self.transform:
            image = self.transform(image)

        label = int(row["label_idx"])
        skin_idx = int(row["skin_idx"])

        return {
            "image": image,
            "label": torch.tensor(label, dtype=torch.long),
            "skin_idx": torch.tensor(skin_idx, dtype=torch.long),
        }

```

```

In [6]: train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    )

```

```

    ),
])

eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

image_roots = [
    BASE / "imgs_1",
    BASE / "imgs_2",
    BASE / "imgs_3",
]

full_dataset = FacetDataset(df, image_roots, transform=None)

```

Indexed 31702 image files from roots: [WindowsPath('C:/Users/Lucas/Desktop/Development/FairMachine/Data/imgs_1'), WindowsPath('C:/Users/Lucas/Desktop/Development/FairMachine/Data/imgs_2'), WindowsPath('C:/Users/Lucas/Desktop/Development/FairMachine/Data/imgs_3')]

```

In [7]: N = len(full_dataset)
        n_test = int(TEST_SPLIT * N)
        n_val = int(VAL_SPLIT * N)
        n_train = N - n_test - n_val

        train_ds, val_ds, test_ds = random_split(full_dataset, [n_train, n_val, n_test])

        # Attach transforms per split
        train_ds.dataset.transform = train_transform
        val_ds.dataset.transform = eval_transform
        test_ds.dataset.transform = eval_transform

        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
                                   num_workers=NUM_WORKERS, pin_memory=True)
        val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
                                num_workers=NUM_WORKERS, pin_memory=True)
        test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,
                                  num_workers=NUM_WORKERS, pin_memory=True)

```

```

In [8]: def build_resnet(num_classes):
        model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
        # Replace final layer
        in_features = model.fc.in_features
        model.fc = nn.Linear(in_features, num_classes)
        return model

        model = build_resnet(num_classes).to(device)

        criterion = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(model.parameters(), lr=LR)

```

```

In [9]: def run_epoch(model, loader, optimizer=None):
        is_train = optimizer is not None
        model.train(is_train)

        total_loss = 0.0
        correct = 0
        total = 0

        # per-group correct / total
        group_correct = defaultdict(int)
        group_total = defaultdict(int)

        all_preds = []
        all_labels = []
        all_skin = []

        for batch in tqdm(loader, leave=False):
            images = batch["image"].to(device)
            labels = batch["label"].to(device)
            skin_idx = batch["skin_idx"].to(device) # 0 light, 1 dark, 2 NA

            if is_train:
                optimizer.zero_grad()

            outputs = model(images)
            loss = criterion(outputs, labels)

            if is_train:
                loss.backward()
                optimizer.step()

            batch_size = images.size(0)
            total_loss += loss.item() * batch_size

            preds = outputs.argmax(dim=1)

            correct_batch = (preds == labels)
            correct += correct_batch.sum().item()
            total += batch_size

            # per-skin-group tracking
            for g in (0, 1, 2):
                mask = (skin_idx == g)
                if mask.any():
                    group_total[g] += mask.sum().item()
                    group_correct[g] += correct_batch[mask].sum().item()

            all_preds.append(preds.detach().cpu())
            all_labels.append(labels.detach().cpu())
            all_skin.append(skin_idx.detach().cpu())

        avg_loss = total_loss / total
        acc = correct / total

        # compute per-group accuracy

```

```

group_acc = {}
for g in group_total:
    group_acc[g] = group_correct[g] / group_total[g]

all_preds = torch.cat(all_preds)
all_labels = torch.cat(all_labels)
all_skin = torch.cat(all_skin)

stats = {
    "loss": avg_loss,
    "accuracy": acc,
    "group_acc": group_acc,
    "all_preds": all_preds,
    "all_labels": all_labels,
    "all_skin": all_skin,
}
return stats

```

```

In [ ]: # Simple function to print per-group accuracies
def describe_group_acc(group_acc):
    name_map = {0: "light", 1: "dark", 2: "NA"}
    for g, acc in group_acc.items():
        print(f" {name_map.get(g, g)}: {acc:.4f}")

```

Training and testing model

```

In [11]: from tqdm.auto import tqdm # notebook-friendly

# Sanity: device
print("torch.version:", torch.__version__)
print("cuda available:", torch.cuda.is_available())
print("device:", device)

# DataLoaders - JUPYTER / WINDOWS SAFE
NUM_WORKERS = 0 # this avoids DataLoader deadlocks in notebooks
pin_memory = (device.type == "cuda")

BATCH_SIZE = 64

train_loader = DataLoader(
    train_ds,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=pin_memory,
)

val_loader = DataLoader(
    val_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=pin_memory,
)

```

```

test_loader = DataLoader(
    test_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=pin_memory,
)

best_val_acc = 0.0
best_state_dict = None

for epoch in range(1, NUM_EPOCHS + 1):
    print(f"\nEpoch {epoch}/{NUM_EPOCHS}")

    train_stats = run_epoch(model, train_loader, optimizer)
    print(f"Train loss: {train_stats['loss']:.4f}, acc: {train_stats['accuracy']:.4f}")
    print("  Train per-skin acc:")
    describe_group_acc(train_stats["group_acc"])

    with torch.no_grad():
        val_stats = run_epoch(model, val_loader, optimizer=None)
        print(f"Val  loss: {val_stats['loss']:.4f}, acc: {val_stats['accuracy']:.4f}")
        print("  Val per-skin acc:")
        describe_group_acc(val_stats["group_acc"])

        if val_stats["accuracy"] > best_val_acc:
            best_val_acc = val_stats["accuracy"]
            best_state_dict = model.state_dict()

# Test evaluation
if best_state_dict is not None:
    model.load_state_dict(best_state_dict)

with torch.no_grad():
    test_stats = run_epoch(model, test_loader, optimizer=None)

print("\nFinal Test Results:")
print(f"  Loss: {test_stats['loss']:.4f}, acc: {test_stats['accuracy']:.4f}")
print("  Test per-skin acc:")
describe_group_acc(test_stats["group_acc"])

```

```

torch.version: 2.5.1+cu121
cuda available: True
device: cuda
Final Test Results:
  Loss: 1.4767, acc: 0.6660
  Test per-skin acc:
  light: 0.6653
  dark: 0.6343
  NA: 0.7363

```

Adversarial model training

Now that we understand what the metrics are for the performance of a ResNet model using the facet data set is, we should apply adversarial debiasing and compare the results.

```
In [12]: from torch.autograd import Function

class GradReverse(Function):
    @staticmethod
    def forward(ctx, x, lambd):
        ctx.lambd = lambd
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        # multiply gradient by -lambda on the way back
        return -ctx.lambd * grad_output, None

def grad_reverse(x, lambd=1.0):
    return GradReverse.apply(x, lambd)
```

```
In [13]: import torch.nn as nn
from torchvision import models

class AdversarialResNet(nn.Module):
    def __init__(self, num_classes, num_groups=3, lambda_adv=1.0):
        super().__init__()
        backbone = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

        # everything except final FC
        self.feature_extractor = nn.Sequential(*list(backbone.children())[:-1])
        self.feats_dim = backbone.fc.in_features

        # task head (class1)
        self.classifier = nn.Linear(self.feats_dim, num_classes)

        # adversary head (skin group: light/dark/NA)
        self.adv_head = nn.Linear(self.feats_dim, num_groups)

        # how strong to push the adversarial signal
        self.lambda_adv = lambda_adv

    def forward(self, x):
        # backbone
        features = self.feature_extractor(x)           # [B, C, 1, 1]
        features = features.flatten(1)                 # [B, C]

        # main task logits
        logits_y = self.classifier(features)

        # apply GRL before adversary
        rev_features = grad_reverse(features, self.lambda_adv)
        logits_a = self.adv_head(rev_features)

        return logits_y, logits_a
```

```
In [14]: num_classes = df["label_idx"].nunique()
num_groups = 3 # skin_idx: 0 Light, 1 dark, 2 NA

lambda_adv = 1.0 # you can tune this later (0.1, 0.5, 1.0, 2.0, ...)
model = AdversarialResNet(num_classes, num_groups=num_groups, lambda_adv=lambda_adv)

criterion_y = nn.CrossEntropyLoss() # task loss
criterion_a = nn.CrossEntropyLoss() # adversary loss (race)

optimizer = torch.optim.Adam(model.parameters(), lr=LR)
```

```
In [15]: from collections import defaultdict
from tqdm import tqdm
import torch.nn.functional as F

def run_epoch_adv(model, loader, optimizer=None, alpha_adv=1.0):
    is_train = optimizer is not None
    model.train(is_train)

    total_loss = 0.0
    total_loss_y = 0.0
    total_loss_a = 0.0
    correct = 0
    total = 0

    group_correct = defaultdict(int)
    group_total = defaultdict(int)

    adv_correct = 0
    adv_total = 0

    all_preds = []
    all_labels = []
    all_skin = []

    for batch in tqdm(loader, leave=False):
        images = batch["image"].to(device)
        labels = batch["label"].to(device)
        skin_idx = batch["skin_idx"].to(device) # 0,1,2

        if is_train:
            optimizer.zero_grad()

        logits_y, logits_a = model(images)

        # main task loss
        loss_y = criterion_y(logits_y, labels)

        # adversary loss (only on skin_idx != 2)
        valid_mask = (skin_idx != 2)
        if valid_mask.any():
            loss_a = criterion_a(logits_a[valid_mask], skin_idx[valid_mask])
        else:
            # no valid groups in this batch
            loss_a = torch.tensor(0.0, device=device)
```

```

loss = loss_y + alpha_adv * loss_a

if is_train:
    loss.backward()
    optimizer.step()

batch_size = images.size(0)
total_loss += loss.item() * batch_size
total_loss_y += loss_y.item() * batch_size
total_loss_a += loss_a.item() * batch_size

# task predictions
preds = logits_y.argmax(dim=1)
correct_batch = (preds == labels)
correct += correct_batch.sum().item()
total += batch_size

# per-skin task accuracy
for g in (0, 1, 2):
    mask_g = (skin_idx == g)
    if mask_g.any():
        group_total[g] += mask_g.sum().item()
        group_correct[g] += correct_batch[mask_g].sum().item()

# adversary accuracy (on valid skin_idx)
if valid_mask.any():
    adv_preds = logits_a[valid_mask].argmax(dim=1)
    adv_correct += (adv_preds == skin_idx[valid_mask]).sum().item()
    adv_total += valid_mask.sum().item()

all_preds.append(preds.detach().cpu())
all_labels.append(labels.detach().cpu())
all_skin.append(skin_idx.detach().cpu())

avg_loss = total_loss / total
avg_loss_y = total_loss_y / total
avg_loss_a = total_loss_a / max(1, total)
acc = correct / total

group_acc = {}
for g in group_total:
    group_acc[g] = group_correct[g] / group_total[g]

adv_acc = adv_correct / adv_total if adv_total > 0 else None

all_preds = torch.cat(all_preds)
all_labels = torch.cat(all_labels)
all_skin = torch.cat(all_skin)

return {
    "loss": avg_loss,
    "loss_y": avg_loss_y,
    "loss_a": avg_loss_a,
    "accuracy": acc,
    "group_acc": group_acc,

```

```

    "adv_acc": adv_acc,
    "all_preds": all_preds,
    "all_labels": all_labels,
    "all_skin": all_skin,
}

```

Training and testing model

```

In [17]: best_val_acc = 0.0
best_state_dict = None

alpha_adv = 1.0 # strength of adversarial loss in total objective

for epoch in range(1, NUM_EPOCHS + 1):
    print(f"\nEpoch {epoch}/{NUM_EPOCHS}")

    train_stats = run_epoch_adv(model, train_loader, optimizer, alpha_adv=alpha_adv)
    print(f"Train total loss: {train_stats['loss']:.4f} "
          f"(cls {train_stats['loss_y']:.4f}, adv {train_stats['loss_a']:.4f}), "
          f"acc: {train_stats['accuracy']:.4f}")
    print(" Train per-skin acc:")
    describe_group_acc(train_stats["group_acc"])
    print(f" Train adversary acc (race predictability): {train_stats['adv_acc']:.4f}
          if train_stats["adv_acc"] is not None else " Train adversary acc: N/A")

    with torch.no_grad():
        val_stats = run_epoch_adv(model, val_loader, optimizer=None, alpha_adv=alpha_adv)
        print(f"Val total loss: {val_stats['loss']:.4f} "
              f"(cls {val_stats['loss_y']:.4f}, adv {val_stats['loss_a']:.4f}), "
              f"acc: {val_stats['accuracy']:.4f}")
        print(" Val per-skin acc:")
        describe_group_acc(val_stats["group_acc"])
        print(f" Val adversary acc (race predictability): {val_stats['adv_acc']:.4f}
              if val_stats["adv_acc"] is not None else " Val adversary acc: N/A")

    # Same selection criterion as baseline: best validation accuracy
    if val_stats["accuracy"] > best_val_acc:
        best_val_acc = val_stats["accuracy"]
        best_state_dict = model.state_dict()

# Evaluate on test
if best_state_dict is not None:
    model.load_state_dict(best_state_dict)

with torch.no_grad():
    test_stats = run_epoch_adv(model, test_loader, optimizer=None, alpha_adv=alpha_adv)

print("\nFinal Test Results (Adversarial Model):")
print(f" Total loss: {test_stats['loss']:.4f} "
      f"(cls {test_stats['loss_y']:.4f}, adv {test_stats['loss_a']:.4f}), "
      f"acc: {test_stats['accuracy']:.4f}")
print(" Test per-skin acc:")
describe_group_acc(test_stats["group_acc"])

```

```
print(f" Test adversary acc (race predictability): {test_stats['adv_acc']:.4f}"  
      if test_stats["adv_acc"] is not None else " Test adversary acc: N/A")
```

Final Test Results (Adversarial Model):

Total loss: 2.1045 (cls 1.5688, adv 0.5357), acc: 0.6543

Test per-skin acc:

light: 0.6518

dark: 0.6245

NA: 0.7349

Test adversary acc (race predictability): 0.7896
